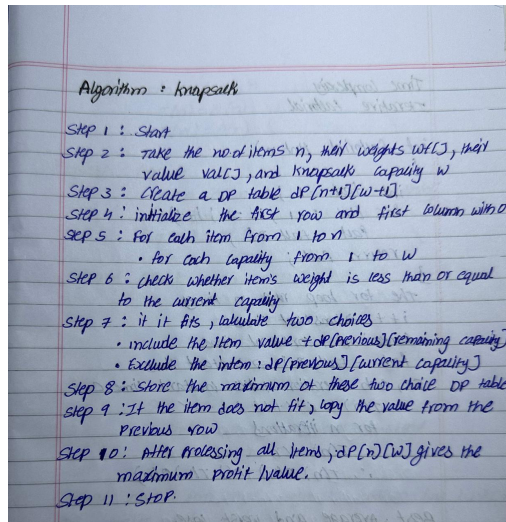




PRACTICAL - 5

AIM : To Implementation of a knapsack problem using dynamic programming.

ALGORITHM // Knapsack



PROGRAM : //

```
#include<iostream>
using namespace std;

class Knapsack {
private:
    int w[10];    // weights
    int v[10];    // values
    int n;        // number of items
    int W;        // capacity
    int dp[10][100]; // dp table

public:
    void getInput() {
        cout << "Enter number of items : ";
        cin >> n;
        cout << "Enter knapsack capacity : ";
```



```
cin >> W;
```

```
    for(int i = 1; i <= n; i++) {  
        cout << "Item " << i << " weight and value : ";  
        cin >> w[i] >> v[i];  
    }  
}
```

```
void solve() {
```

```
    // base case — row 0 and col 0  
    for(int i = 0; i <= n; i++) dp[i][0] = 0;  
    for(int j = 0; j <= W; j++) dp[0][j] = 0;
```

```
    // fill table
```

```
    for(int i = 1; i <= n; i++) {  
        for(int j = 1; j <= W; j++) {
```

```
            if(w[i] <= j) {
```

```
                dp[i][j] = max(  
                    v[i] + dp[i-1][j-w[i]],  
                    dp[i-1][j]  
                );
```

```
            } else {
```

```
                dp[i][j] = dp[i-1][j];
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
void printTable() {
```

```
    cout << "\nDP Table:" << endl;
```

```
    cout << "    ";
```

```
    for(int j = 0; j <= W; j++)
```

```
        cout << j << "    ";
```



```
cout << endl;

for(int i = 0; i <= n; i++) {
    cout << "i=" << i << " ";
    for(int j = 0; j <= W; j++)
        cout << dp[i][j] << " ";
    cout << endl;
}
}

void findItems() {
    cout << "\nSelected Items:" << endl;

    int j = W;

    for(int i = n; i >= 1; i--) {
        if(dp[i][j] != dp[i-1][j]) {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") TAKEN " << endl;
            j = j - w[i];
        } else {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") NOT taken " << endl;
        }
    }
}

void showResult() {
    cout << "\nMaximum Value = "
```

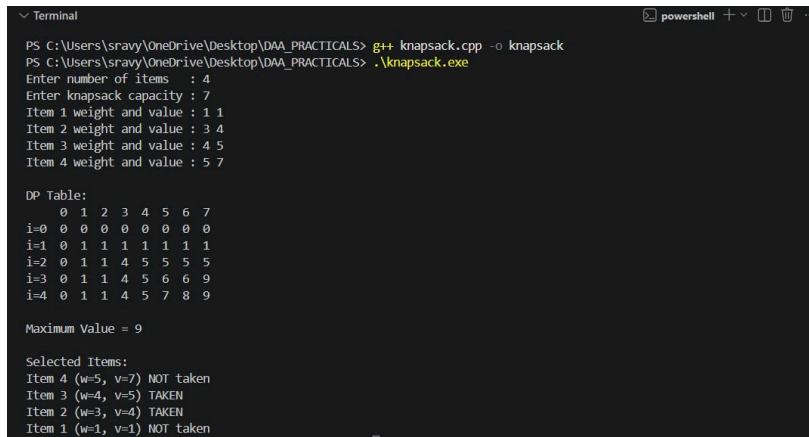
```
<< dp[n][W] << endl;
    }
};

int main() {

    Knapsack k;

    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();
    return 0;
}
```

OUTPUT :



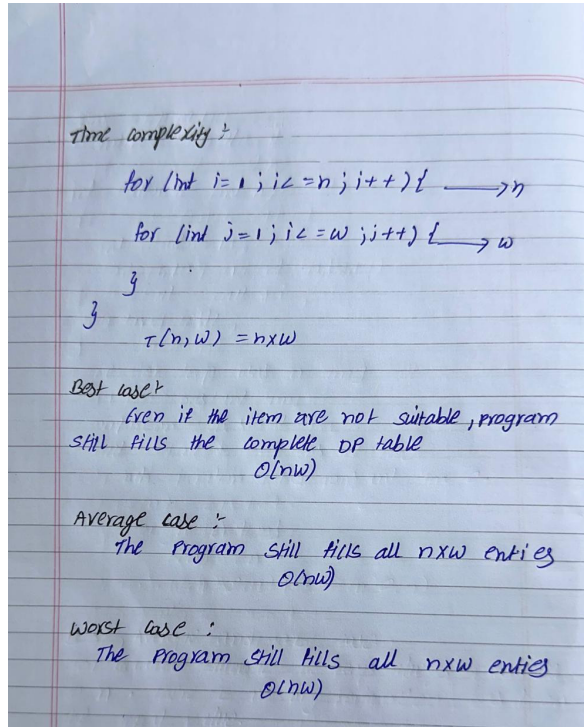
```
PS C:\Users\sra\OneDrive\Desktop\DAA_PRACTICALS> g++ knapsack.cpp -o knapsack
PS C:\Users\sra\OneDrive\Desktop\DAA_PRACTICALS> .\knapsack.exe
Enter number of items : 4
Enter knapsack capacity : 7
Item 1 weight and value : 1 1
Item 2 weight and value : 3 4
Item 3 weight and value : 4 5
Item 4 weight and value : 5 7

DP Table:
  0 1 2 3 4 5 6 7
i=0 0 0 0 0 0 0 0
i=1 0 1 1 1 1 1 1
i=2 0 1 1 4 5 5 5
i=3 0 1 1 4 5 6 6
i=4 0 1 1 4 5 7 8 9

Maximum Value = 9

Selected Items:
Item 4 (w=5, v=7) NOT taken
Item 3 (w=4, v=5) TAKEN
Item 2 (w=3, v=4) TAKEN
Item 1 (w=1, v=1) NOT taken
```

Time Complexity:



Time complexity :-
for (int i=1; i<=n; i++) { $\rightarrow n$
for (int j=1; j<=W; j++) { $\rightarrow W$
}
}
 $T(n, W) = n \times W$

Best case :-
Even if the items are not suitable, program still fills the complete DP table
 $O(nW)$

Average case :-
The program still fills all $n \times W$ entries
 $O(nW)$

Worst case :-
The program still fills all $n \times W$ entries
 $O(nW)$

Conclusion : The 0/1 Knapsack Problem was successfully implemented using Dynamic Programming. The algorithm efficiently finds the maximum value that can be obtained within the given knapsack capacity by considering whether each item should be selected or not selected. The time complexity of the algorithm is $O(nW)$ and the space complexity is $O(nW)$